

Assignment 4

1 Instructions

- **Deadline:** 24 hours before next laboratory class.
- Read the following documentation and, **first of all**, draw **UML Class Diagram** for both projects. Save them as image (*.png, *.jpg, *.jpeg) and **copy each one into the root of the corresponding project**.
- Rename projects (Refactor → Rename) as `surname1_surname2_name_lab04_dome` and `surname1_surname2_name_lab04_socialnetwork`, lowercase, without accents. Then, export them to a **single** ZIP file named `surname1_surname2_name_lab04` and upload it to the corresponding task in Campus Virtual.
- Rename package names `.lab03.` to `.lab04..`

Whatever the exercise, be sure you **always meet all the following rules**:

1. Unless other thing is stated, null are considered **invalid arguments** as well as empty (length 0) and blank (consists of whitespace) strings.
2. Write **tests first**.
3. When **updating** a project, check that existing methods keep working properly. Even more, include new characteristics in existing tests.
4. Review the **checklist** to check the correctness of your project according to the learning outcomes.

2 Project Dome

From now on, items will have a **base price** in range `[0.0 – 1000.0]`. Pass `double` as last parameter to the constructors and throw `IllegalArgumentException` if it is **out of bounds**.

`MediaLibrary` should be able to determine the **total price** of all items in the library.

In addition, we also want class `MediaLibrary` to be able to generate a sequence of **characters summarizing all items in the library**.

Each item will be represented in this sequence by a **code** consisting on part of the title plus an increasing number ≥ 0 , different for each one.

2.1 Testing

Write tests for `equals` or `toString` **only if** you implement them **by hand (no wizard)**.

Then, read next sections and **before implementing**, write functional and robustness tests for new public methods in `MediaLibrary`.

Use `Assertions.assertEquals(double expected, double actual, double delta)` for asserting doubles.

Since doubles may not be exactly equal `delta` lets you describe how close they have to be to pass.

If you were expecting 3.14159 but would take any from 3.14059 to 3.14259 (that is, within 0.001), then you should write something like

```
assertEquals(3.14159, 3.1421, 0.001)
```

2.2 `public boolean equals(Object o)`

Be sure every item implement it, the hard way (by hand) or using the wizard.

Remember conditions for equality. Two items are equal when:

- a They are both null
- b They are the same object reference
- c They are the same type, have the same title and

Cd same artist

Dvd same director

VideoGame same author and platform

2.3 `public double getTotalPrice()`

Implement the following method in `MediaLibrary`.

```
/**
 * @return total price of all items in the library.
 *         It is calculated as the sum of the final prices of all items
 */
public double getTotalPrice()
```

The **final price of each item** is calculated as follows:

- For CD, increases based price by a fixed tax of 2.0 euros.
- For DVD, the final price is the base price.
- For video games, increases base price by 10%.

2.4 public String generateCodes()

Implement the following method in class `MediaLibrary`:

```
/**
 * @return This method returns a string as the result of concatenating
 *         item's code, joined by a dash (-).
 *         If there is no item, returns an empty string.
 */
public double generateCodes()
```

The **code of each item** will be a string consisting on the first three letters of the title (if there are enough, otherwise, return whatever is) plus its position in the library. It will be 0 for first item, 1 for second is 1, and so on.

For example, if the title of the first item in `MediaLibrary` is *Yesterday* and the second one is *All You Need Is Love*, the result will be *Yes0-All1*.

You might need to use method `String.substring(...)`.

3 Extending the social network

3.1 public List<String> toHtmlFormat()

Implement the following method to class `SocialNetwork`

```
/**
 * @return This method will return a list of strings.
 *         Each one is for a post, in the same order than in the library.
 */
public List<String> toHtmlFormat()
```

Every post is formatted in HTML, according to the following:

Text post will be formatted as `<p>message</p>`

Image post will be done as `the caption</img>`

Do as in `Dome`, overriding `toString` in subtypes. Use `toString` instead of `print`.

3.2 NetworkApp::simulateClient()

Modify the method `main` for printing out all the posts in HTML (one per line).

Sample output:

Posts in HTML format:

```
<img src = photo1> The caption </img>
<p> I am going to the cinema </p>
<img src = photo2> Other caption </img>
<p> I am in Africa </p>
```